

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

ваши вопросы

Битовые операторы

- применяются к целым числам
- унарные
 - NOT (НЕ)
- бинарные
 - AND (И)
 - OR (ИЛИ)
 - XOR (ИСКЛЮЧАЮЩЕЕ ИЛИ)
 - сдвиг

Биты как элементы множества

- NOT – дополнение
- AND – пересечение
- OR – объединение
- XOR – симметрическая разность

Создание масок

- константы (литералы, макросы)
- сдвиги и др. операторы

пример:

```
const uint32_t m1 (1 << 4);  
uint32_t m2 (0xFF0B1000);
```

Работа с масками

- AND
 - сброс битов по маске
- OR
 - установка битов по маске
- XOR
 - для инвертирования битов по маске

Битовое НЕ (~)

инверсия битов

пример:

```
uint32_t v(15 + 11 * 256);  
printf("%x ", v);  
printf("%x ", ~v);  
v = ~v;  
printf("%x\n", v);
```

> b0f ffffffff4f0 f4f0

Битовое И (&)

совпадение ненулевых битов

пример:

```
uint32_t v1(15 + 11 * 256);  
uint32_t v2(9 + 11 * 256);  
printf("%x & %x = %x\n", v1, v2, v1 & v2);
```

> 00000b0f & 00000d0b = 0000090b

Битовое ИЛИ (|)

ненулевые биты

пример:

```
uint32_t v1(15 + 11 * 256);
```

```
uint32_t v2(9 + 11 * 256);
```

```
printf("%x | %x = %x\n", v1, v2, v1 | v2);
```

> 00000b0f | 00000d0b = 00000f0f

Битовое ИСКЛЮЧАЮЩЕЕ ИЛИ (^)

несовпадение ненулевых битов

пример:

```
uint32_t v1(15 + 11 * 256);  
uint32_t v2(9 + 11 * 256);  
printf("%x ^ %x = %x\n", v1, v2, v1 ^ v2);
```

> 00000b0f ^ 00000d0b = 00000604

БИТОВЫЙ СДВИГ

- смещение бит в указанном направлении на указанное число позиций
- «вылезавшие» за пределы числа биты теряются
- «дырки» заполняются

БИТОВЫЙ СДВИГ ВЛЕВО (<<)

правые биты заполняются нулями

пример:

```
int32_t  si(0xFFFFFFFF);  
uint32_t ui(0xFFFFFFFF);  
printf("%08x %08x", si << 4, ui << 2)
```

> ffffffff0 ffffffff8

Битовый сдвиг вправо (>>)

- левые биты заполняются
 - нулями для беззнаковых
 - битом знака для знаковых (зависит от реализации)

пример:

```
int32_t  si(0xFFFFFFFF);  
uint32_t ui(0xFFFFFFFF);  
printf("%08x %08x", si >> 4, ui >> 4)
```

> ffffffff 0fffffff

Присваивающие версии операторов

- `lhs &= rhs`
- `lhs |= rhs`
- `lhs ^= rhs`

- `val <<= shift`
- `val >>= shift`

ваши вопросы

ПОТОКОВЫЙ ВЫВОД И ВВОД

- вывод – преобразование объектов (типа) в последовательность символов
- ввод – запрос последовательности символов и преобразование ее в объект (типа)

Тип потока (базовый класс)

- `ostream` – вывода (выходной)
- `istream` – ввода (входной)
- `iostream` – ввода и вывода
(двунаправленный)

Заголовочные файлы

`<iostream>`

– стандартные потоки

`<fstream>`

– файловые потоки

`<sstream>`

– строковые потоки

`<iosfwd>`

– опережающие
объявления

`<iomanip>`

– манипуляторы

`<streambuf>`

– буферы

Преимущества стандартной библиотеки ввода/вывода

- контроль типов передаваемых данных
- использование одного кода для работы с разными потоками (через полиморфизм)
- расширяемость
 - ВВОД И ВЫВОД НОВЫХ ТИПОВ
 - НОВЫЕ КЛАССЫ ПОТОКОВ

Стандартные потоки

- глобальные переменные
- возможно перенаправление
- консольный ввод и вывод (`cin` и `cout`)
- протоколирование ошибок (`cerr`)
- ведение логов (`clog`)

Опережающее объявление

- сокращает время компиляции
- можно использовать для указателей и ссылок

пример:

```
// in comp.h
```

```
class Stack;
```

```
...
```

```
bool compute(const Stack& stack);
```

Опережающее объявление потоков

```
// class ostream; // не работает

namespace std {
    template<
        class Ch,
        class Tr = char_traits<Ch> >
        class basic_ostream;

    typedef basic_ostream<char> ostream;
}
```

Базовые операции

- открытие (создание)
- `operator<<`
 - запись в поток (поместить данные)
- `operator>>`
 - чтение из потока (извлечь данные)
- проверка состояния
- закрытие (уничтожение)

Типы ввода и вывода

- форматированный
 - текстовый
- неформатированный
 - двоичный

Формат

- система счисления (oct, dec, hex)
- отступы
- заполнители
- разделители
- формат префиксов и суффиксов
- разделителя разрядов
- и т.д.

Форматирование

- формат – состояние потока
(флаги форматирования)
- изменение формата
 - функции
 - манипуляторы

Форматные флаги (примеры)

- `ios::showbase`
 - признак системы счисления (dec, oct или hex)
- `ios::showpoint`
 - десятичная точка и завершающих нулей в вещественных числах
- `ios::uppercase`
 - верхний регистр букв A-F (hex) и E (exp)
- `ios::showpos`
 - отображение знака плюс (+) для положительных величин

Флаги форматирования (функции)

`fmtflags flags()`

- получить текущие флаги

`fmtflags flags(fmtflags flNew)`

- установить флаги

`fmtflags setf(fmtflags flSet)`

`fmtflags setf(fmtflags bits,
fmtflags field)`

- добавить флаги

`fmtflags unsetf(fmtflags flClear)`

- снять флаги

Форматное поле

- должен быть установлен только один бит из группы
- автоматический сброс остальных бит не происходит
- используйте перегруженную версию `setf`

Форматные поля

- система счисления целых

`ios::basefield`

- вид чисел с плавающей точкой

`ios::floatfield`

- выравнивание внутри поля

`ios::adjustfield`

Форматные поля (примеры)

- `ios::oct`
– восьмиричные целые
- `ios::dec`
– десятичные целые
- `ios::hex`
– шестнадцатиричные целые

пример:

```
s.setf(ios::hex, ios::basefield);
```

Манипулятор

- специальный объект, вызывающий функцию потока
- стандартные манипуляторы определены в пространстве имен `std`

пример:

```
strm << hex << val << endl;
```


Цикл работы с форматами

- аналогичен работе с ресурсами
- запомнить текущее состояние
- произвести всю необходимую работу
- восстановить исходное состояние

Буферизация потоков

- для записи и чтения используется буфер, который освобождается при переполнении или по требованию

пример:

```
strm.flush();
```

```
strm << obj << endl;
```

Синтаксис форматированного ВВОДА И ВЫВОДА

перегруженные операторы двоичного сдвига

```
cout << "a=" << a;
```

означает

```
operator<<(cout, "a=") .operator<<(a) ;
```

Ввод и вывод

ПОЛЬЗОВАТЕЛЬСКИХ ТИПОВ

```
ostream& Num::writeTxt(ostream& ostrm) {  
    ostrm << '(' << n_ << ')';  
    return ostrm;  
}
```

```
ostream& operator<<(ostream& ostrm,  
    const Num& num) {  
    return num.writeTxt(ostrm);  
}
```

Построчный ввод

для чтения текстового файла по строкам
используйте функцию `getline`
определенную в `<string>`

пример:

```
while (0 != getline(file, str).fail()) {  
    // ...  
}
```

Обработка ошибок ввода и вывода

- существует состояние потока
- текущее состояние зависит от успешности предыдущих операций с потоком
- возможность реализации операции с потоком зависит от текущего состояния

Состояние потока (флаги)

- `ios::badbit`
 - фатальная ошибка, поток использовать нельзя
- `ios::eofbit`
 - обнаружен конец входных данных (физически достигнут конец файла, пользователь закончил консольный ввод нажатием Ctrl+Z или Ctrl+D)

Состояние потока (флаги)

- `ios::failbit`
 - операция завершилась неудачей (нарушен формат или обнаружен конец данных)
- `ios::goodbit`
 - ошибок нет, конец данных не обнаружен

Проверка состояния потока

- `good()`
 - `goodbit`
- `eof()`
 - `eofbit` **или** `badbit`
- `fail()`
 - `failbit` **или** `badbit`
- `bad()`
 - `badbit`

Установка состояния потока

- `clear(iostate _State = goodbit)`
– сбросить все флаги и установить заданные
- `setstate(iostate _State)`
– установить флаги

пример:

```
cout.clear(ios::failbit);  
cout.setstate(ios::failbit | ~ios::goodbit);
```

Исключения в потоках

- поток может генерировать исключения `std::ios_base::failure` для заданных флагов состояния
- `exceptions`
 - устанавливает статусы

пример:

```
stmFile.exceptions (ios::badbit) ;
```

Неформатированный ввод

- чтение отдельного символа или последовательности из потока
- `get`
 - прочитать из потока отдельный символ
- `read`
 - прочитать из потока последовательность

Неформатированный вывод

- вывод отдельного символа или последовательности в поток
- `put`
 - записать в поток отдельный символ
- `write`
 - записать в поток последовательность

ваши вопросы

Файловые потоки (<fstream>)

- прикрепляются к объектам файловой системы
- `ofstream` - ВЫВОД
- `ifstream` - ВВОД
- `fstream` - ВВОД И ВЫВОД

Цикл работы с файлом

- конструктор
- `open`
- `close`
- деструктор

пример:

```
ifstream inData(path) ;
```


Режим открытия файла (флаги)

- `ios::in`
– чтение (без усечения)
- `ios::out`
– запись (без усечения)
- `ios::app`
– дописывание

Режим открытия файла (флаги)

- `ios::ate`
– существующий файл (чтение, запись в конец)
- `ios::trunc`
– существующий файл (запись с начала)
- `ios::binary`
– двоичный режим

Файловый поток (пример)

```
fstream dataFile(path, ios::ate);  
dataFile << obj1;  
dataFile << obj2;  
dataFile.close();  
  
...  
dataFile.open(path, ios::in);  
  
...
```

Файловый поток (пример)

```
// s - stream, v - vector  
s.write(reinterpret_cast<char*>(&len_),  
        sizeof(len_));  
s.write(reinterpret_cast<char*>(v.data()),  
        sizeof(v[0]) * v.size());  
  
s.read(reinterpret_cast<char*>(&len_),  
        sizeof(len_));
```

Позиционирование в потоке

- `std::streamsize` – позиция
- `tellp` – получить позицию записи
- `tellg` – получить позицию чтения
- `seekp` – установить позицию записи
- `seekg` – установить позицию чтения

Относительное позиционирование

- `ios_base::seekdir` – позиционирование
- `ios::beg` – от начала
- `ios::cur` – от текущей позиции
- `ios::end` – от конца

Позиционирование (пример)

```
ifstream is("data.txt", ios_base::binary);  
if (is) {  
    is.seekg(0, is.end); // размер файла  
    ptrdiff_t fileSize(is.tellg());  
    is.seekg(0, is.beg);  
    ...  
}
```

Строковые потоки

- прикрепляются к строковым объектам
- `ostream`
- `istream`
- `stringstream`
- `str()` – получение буфера `string`

Строковый поток (пример)

```
ostreamstream strm(strBuf);  
strm << "Formatted text\n";  
...
```

```
istreamstream strm(strBuf);  
strm >> obj;  
...
```

ваши вопросы

Умолчательные параметры функции (необязательные параметры)

- последние параметры в списке могут иметь умолчательные инициализаторы
- вычисляются на этапе компиляции
- не должны мешать разрешению перегрузки

Умолчательные параметры функции (пример)

```
void f(int a, int b = 0, int c = 0);
```

```
f(z);
```

```
f(y, z);
```

```
f(y, z, z);
```

ваши вопросы